Automation Template Selenium Java Cucumber

Manual de uso

Template reutilizable de automatización Web UI

v1.3.0 — Docker & CI Integration
Stable / Validated / Published
Fecha de lanzamiento: 12/10/2026
Java 21 (predeterminado) | Java 17 (compatible)
Selenium 4.48.0
Cucumber 7.34.7
JUnit 5.13.4
Gradle 8.14.5
SLF4J 2.0.20 | Logback 1.6.5

Índice

1. Inicio rápido para primera vez
2. Arquitectura
3. Stack, requisitos y VS Code
3.1 Configurar Java 17 paso a paso
4. Configuración y comandos
4.1 Uso con Docker
5. Primera automatización
6. Reutilización
7. Contrato CI/CD, buenas prácticas y soporte
8. CI/CD con GitHub Actions
9. Cómo ejecutar y revisar el CI/CD
10. Quality Gate, fallos y evidencias
11. Protección de la rama main
12. Logging y observabilidad básica
12.1 Flujo de observabilidad
12.2 Niveles de logging
12.3 Ejecución y seguridad
12.4 Diagnóstico del logging
13. Captura automática de evidencias
14. Generación y consulta de reportes
15. Glosario

Propósito

Esta guía permite instalar, configurar, ejecutar y ampliar el template, interpretar resultados y resolver fallos. Explica desde cero el flujo Git, Pull Request y GitHub Actions. Para crear pruebas se necesitan conocimientos básicos de Java. El ejemplo incluido es neutral y usa una página HTML local.

Información del documento

Campo	Valor
Autor / Creador del template	Israel Baz
Rol	Test Automation / Prompt Engineering / AI Automation
Mantenimiento	Israel Baz

Estado e historial de versiones

Versión	Fecha / estado	Cambios principales
v1.3.0	2026-10-12 UTC Published	Docker & CI Integration.
v1.2.0	8 de octubre de 2026	Stable / Validated / Published. Logging con SLF4J/Logback, evidencias automáticas, reportes HTML/JSON y mejoras de estabilidad.
v1.1.0	1 de octubre de 2026	Stable / Validated / Published. CI con Quality Gate y artifacts de resultados.
v1.0.2	26 de septiembre de 2026	Documentation-only Hotfix, Stable / Validated / Published.
v1.0.1	25 de septiembre de 2026	Hardening + CI/CD Readiness. Stable / Validated / Published.
v1.0.0	18 de septiembre de 2026	Primera versión estable: Java, Selenium, Cucumber, POM y manual PDF.

1. Inicio rápido para primera vez

Siga estos pasos sin modificar ningún archivo. Si un comando termina con BUILD SUCCESSFUL, se ejecutó correctamente. Si termina con BUILD FAILED, conserve el mensaje y consulte Troubleshooting antes de cambiar código.

Paso 1 - Abra PowerShell y navegue a la raíz:

```
cd C:\ruta\automation-template-selenium-java-cucumber
```

Paso 2 - Ejecute la prueba de ejemplo sin ventana visible:

```
.\gradlew.bat test -Dheadless=true
```

Paso 3 - Abra el reporte de ejecución:

Abra build/reports/cucumber/cucumber.html. El escenario abre la fixture HTML local src/test/resources/fixtures/example_page.html y valida el encabezado Example Domain. La fixture está versionada y no depende del contenido de un sitio externo; el escenario se reproduce en CI.

Paso 4 - Ejecute con navegador visible:

Ejecute .\gradlew.bat clean test sin el parámetro headless.

Cómo comprobar VS Code

Debe ver src, gradle, build.gradle y settings.gradle. Abra example_domain.feature: debe aparecer coloreado como Gherkin. Abra ExampleDomainPage.java: Java no debe mostrar imports en rojo. Si VS Code solicita un JDK, seleccione Java 21 y espere que termine "Importing Gradle project".

Lista antes de pedir ayuda

Estoy en la raíz del proyecto; java -version muestra Java 21; el navegador está instalado; la fixture local existe; la Feature termina en .feature y está bajo src/test/resources/features; cada frase Gherkin coincide con su Step; no guardé credenciales ni tokens en Git.

2. Arquitectura

El template estandariza la automatización Web UI: configuración externa, navegador, ejecución BDD, Page Object Model, evidencia y reportes. Es adecuado para proyectos que operan en Chrome o Edge. La configuración tiene prioridad JVM (-D), variable de entorno y archivo config.properties.

Arquitectura general

Capa	Responsabilidad
Feature	Describe comportamiento en Gherkin.
Step Definitions	Traduce intención a llamadas del framework.
Page Objects	Encapsula locators, esperas e interacciones.
WebDriver	Controla Chrome o Edge.
Hooks / reports	Gestiona ciclo de vida, screenshots y resultados.

Flujo de ejecución

gradlew test -> Gradle compila -> JUnit Platform descubre el Runner -> Cucumber carga Features -> Hook Before crea WebDriver -> Steps llaman Pages -> Hook After captura evidencia y cierra -> Reporte HTML.

Regla de diseño: Steps no contienen selectores; Pages no contienen aserciones de escenario; Hooks no incluyen lógica de negocio.

Estructura principal: src/main/java/com/automation/template/{config,driver,pages}; src/test/java/com/automation/template/{hooks,runners,steps,support}; src/test/resources/{features,fixtures,config.properties}. El workflow está en .github/workflows/ci.yml y el manual se genera con scripts/create_manual.py.

3. Stack, requisitos y VS Code

Tecnología	Versión	Uso
Java	21 (pred.) / 17	Lenguaje y toolchain.
Gradle Wrapper	8.14.5	Build y dependencias reproducibles.
Selenium Java	4.48.0	WebDriver.
Cucumber Java/Engine	7.34.7	BDD y Gherkin.
JUnit BOM	5.13.4	JUnit Platform y Suite.
SLF4J API	2.0.20	Fachada de logging.
Logback Classic	1.6.5	Proveedor de logs durante las pruebas.

v1.3.0 identifica el template; 17/21 son versiones de Java seleccionables, y las demás cifras son versiones de dependencias fijadas en build.gradle y gradle-wrapper.properties.

Prerrequisitos

Instale JDK 21 (recomendado) o JDK 17 (compatible), y Chrome o Edge. Use siempre el Gradle Wrapper incluido; en Windows, .\gradlew.bat. No requiere Gradle global. Valide con:

```
java -version
.\gradlew.bat --version
git --version
```

Apertura en Visual Studio Code

Abra PowerShell y ejecute:

```
cd C:\ruta\automation-template-selenium-java-cucumber
code .
```

Instale Extension Pack for Java, Gradle for Java, Cucumber (Gherkin) Full Support y GitLens. Espere que VS Code importe Gradle. Las Features están en src/test/resources/features; Pages en src/main/java; Steps, Hooks y Runner en src/test/java.

Selección de versión de Java

Java 21 es el predeterminado de la configuración actual. Java 17 es la única compatibilidad alternativa mediante la propiedad Gradle javaVersion. Java 18, 19, 20, 22 y cualquier otro valor distinto de 17 o 21 se rechazan explícitamente.

```
.\gradlew.bat clean test
.\gradlew.bat clean test -PjavaVersion=17
.\gradlew.bat clean test -PjavaVersion=21
.\gradlew.bat clean test -PjavaVersion=18 # rechazado
```

Gradle debe ejecutarse con JDK 17 o superior y la toolchain seleccionada debe estar instalada o disponible para Gradle.

Sección 3.1 — Configurar Java 17 paso a paso

Use Java 17 solo cuando lo requiera su proyecto. Java 21 continúa siendo el predeterminado de la configuración actual.

1. Instale un JDK 17 aprobado por su equipo. En estos ejemplos, la carpeta se representa como C:\ruta\jdk-17.

2. Abra PowerShell en la raíz del proyecto y configure la sesión actual. Esto no modifica permanentemente Windows:

```
$env:JAVA_HOME = 'C:\ruta\jdk-17'
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
java -version
```

3. Confirme que java -version indica Java 17. En VS Code, use Ctrl+Shift+P, ejecute Java: Configure Java Runtime, seleccione JDK 17 y espere la importación Gradle.

4. Ejecute las pruebas:

```
.\gradlew.bat clean test -PjavaVersion=17
```

5. Confirme BUILD SUCCESSFUL y abra build/reports/cucumber/cucumber.html. Para volver a Java 21, abra una consola nueva o cambie JAVA_HOME a C:\ruta\jdk-21 y ejecute -PjavaVersion=21.

No guarde rutas de JDK ni JAVA_HOME dentro del repositorio. El workflow incluido usa Java 21. Si adopta Java 17 en otro proyecto, adapte y valide su CI antes de usarlo.

4. Configuración y comandos

config.properties contiene los valores por defecto. Los valores pueden reemplazarse con -D o con variables BASE_URL, BROWSER, HEADLESS, TIMEOUT_SECONDS y SCREENSHOT_ON_FAILURE. javaVersion usa -P, no -D.

```
browser=CHROME
headless=false
baseUrl=https://example.com/
timeoutSeconds=15
screenshotOnFailure=true
```

Objetivo	Comando
Limpiar	.\gradlew.bat clean
Limpiar y ejecutar	.\gradlew.bat clean test
Headless	.\gradlew.bat clean test -Dheadless=true
Chrome	.\gradlew.bat clean test -Dbrowser=CHROME
Edge	.\gradlew.bat clean test -Dbrowser=EDGE
URL	.\gradlew.bat clean test -DbaseUrl=https://su-aplicacion
Tags	.\gradlew.bat clean test -Dcucumber.filter.tags=@example
Tags headless	.\gradlew.bat clean test -Dheadless=true -Dcucumber.filter.tags=@example
Alias Cucumber	.\gradlew.bat cucumber

El smoke test incluido usa la fixture HTML local versionada y no depende de un sitio externo. baseUrl sigue disponible para los Page Objects de aplicaciones reales, pero no modifica este escenario de ejemplo. Los reports se generan en build/reports/cucumber/cucumber.html, build/reports/cucumber/cucumber.json, build/reports/tests/test y build/test-results/test; el log se guarda en build/logs/automation.log y las capturas de fallos en build/evidence/screenshots sólo si falla un escenario, screenshotOnFailure está activo y WebDriver permite capturar.

4.1. Uso con Docker

Docker ejecuta el framework en un contenedor Linux reproducible, separado del Java y navegador de Windows. La versión publicada v1.3.0 contiene Temurin 21, Chrome y ChromeDriver 155.0.8059.39, Gradle Wrapper 8.14.5, Selenium, Cucumber, JUnit y POM. Su estado es Stable / Validated / Published desde el 12 de octubre de 2026 UTC (ISO: 2026-10-12).

Descargar el proyecto desde GitHub

Instale Git para Windows desde git-scm.com/download/win y abra PowerShell. Clone el código fuente, entre al directorio y consulte las ramas remotas:

```
git --version
git clone https://github.com/IsraelBaz-coder/selenium-java-cucumber-automation-template.git
Set-Location .\selenium-java-cucumber-automation-template
git fetch origin
git branch -r
```

Seleccione una referencia que incluya Docker, como main. Compruebe los tres archivos:

```
Test-Path .\Dockerfile
Test-Path .\gradlew
Test-Path .\build.gradle
git check-attr eol -- gradlew
```

Los tres Test-Path deben ser True y Git debe mostrar gradlew: eol: lf. La corrección desde d173e7d forma parte de main.

git clone descarga fuentes; docker build crea la imagen localmente; docker pull requeriría una imagen publicada en un registro. Aún no hay una imagen preconstruida publicada y verificada.

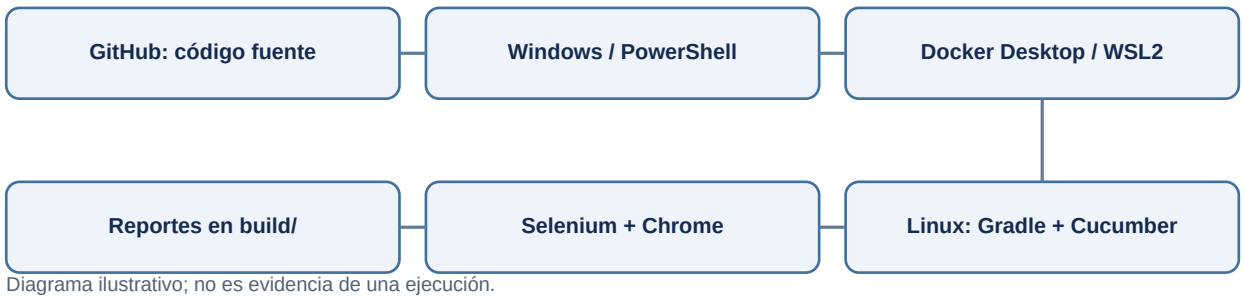


Diagrama ilustrativo; no es evidencia de una ejecución.

Descripción del diagrama: GitHub entrega el código a Windows; PowerShell solicita a Docker Desktop/WSL2 iniciar el contenedor; Gradle y Cucumber ejecutan Selenium con Chrome; los resultados se escriben bajo build/.

1. Windows, WSL2 y Docker Engine

Objetivo: Preparar el motor Linux. **Requisitos:** Docker Desktop instalado, WSL2 y virtualización disponibles.

```
wsl --status
docker version
docker info
```

Explicación: Abra Docker Desktop desde Inicio y espere a que el motor esté activo; Client y Server deben aparecer. **Resultado esperado:** Server muestra Docker Desktop y Linux.

Cómo reconocer un error: No aparece Server o se informa daemon unavailable. **Cómo resolverlo:** Inicie o reinicie Docker Desktop, compruebe modo Linux; si WSL falla, ejecute wsl --update y reinicie cuando se solicite.

2. PowerShell en la carpeta correcta

Objetivo: Usar este Dockerfile como contexto. **Requisitos:** Repositorio descargado en Windows.

```
Get-Location
Test-Path .\Dockerfile
Test-Path .\gradlew
Test-Path .\build.gradle
git check-attr eol -- gradlew
```

Explicación: Trabaje desde la carpeta clonada después de seleccionar una referencia con Docker. **Resultado esperado:** Los tres Test-Path devuelven True y eol indica lf.

Cómo reconocer un error: False o eol unspecified indica otra carpeta o una revisión sin la corrección. **Cómo resolverlo:** Abra la carpeta correcta y use una revisión que incluya .gitattributes con gradlew text eol=lf.

3. Construir la imagen

Objetivo: Crear la imagen de desarrollo. **Requisitos:** Docker Engine activo, raíz correcta y red para descargas.

```
docker build --progress=plain -t selenium-java-cucumber-template:1.3.0-dev .
```

Explicación: --progress=plain muestra cada paso, -t asigna nombre y etiqueta, el punto usa la carpeta actual. **Resultado esperado:** El build termina sin ERROR y deja disponible la imagen local.

Cómo reconocer un error: ERROR o código distinto de cero. **Cómo resolverlo:** Revise daemon, red, proxy y certificados. Ante TLS/PKIX no desactive TLS.

4. Verificar componentes

Objetivo: Inspeccionar versiones dentro de la imagen. **Requisitos:** Imagen construida.

```
docker run --rm selenium-java-cucumber-template:1.3.0-dev java -version
docker run --rm selenium-java-cucumber-template:1.3.0-dev google-chrome --version
docker run --rm selenium-java-cucumber-template:1.3.0-dev chromedriver --version
```

Explicación: --rm elimina cada contenedor temporal; el argumento final reemplaza el comando de pruebas. **Resultado esperado:** Java 21; Chrome y ChromeDriver 155.0.8059.39.

Cómo reconocer un error: Imagen ausente o versiones de navegador y driver distintas. **Cómo resolverlo:** Reconstruya desde la raíz y confirme CHROME_VERSION en Dockerfile.

5. Ejecutar e interpretar

Objetivo: Ejecutar el smoke test headless. **Requisitos:** Imagen disponible.

```
docker run --rm selenium-java-cucumber-template:1.3.0-dev
```

Explicación: Sin otro comando, se ejecuta ./gradlew --no-daemon test. Sin montaje, build/ desaparece con el contenedor. **Resultado esperado:** WebDriver initialized successfully, Example Domain PASSED y BUILD SUCCESSFUL.

Cómo reconocer un error: BUILD FAILED o paso rojo; WARN por sí solo no es fallo. **Cómo resolverlo:** Lea el primer error real. TECH-001 (CDP) sigue OPEN; TECH-002 (selector) quedó CLOSED.

6. Persistir reportes

Objetivo: Guardar build/ en Windows tras --rm. **Requisitos:** Carpeta compartida y escritura para UID 10001.

```
New-Item -ItemType Directory -Force .\build | Out-Null
$out = (Resolve-Path .\build).Path
$bind = "type=bind,source=$out,target=/home/automation/app/build"
docker run --rm --mount $bind selenium-java-cucumber-template:1.3.0-dev
```

Explicación: El bind mount enlaza build/ del host con el contenedor para conservar HTML, JSON, XML y log en Windows. **Resultado esperado:** BUILD SUCCESSFUL y archivos en build/reports/ y build/logs/. Un PNG de fallo no se validó porque el escenario pasó.

Cómo reconocer un error: Permission denied o reportes ausentes. **Cómo resolverlo:** Compruebe la carpeta, Docker Desktop y permisos para UID 10001; consulte docs/DOCKER.md.

Errores y advertencias

Si ./gradlew: not found aunque existe, compruebe git check-attr eol -- gradlew: debe ser lf; chmod +x no corrige CRLF. Daemon detenido: inicie Docker Desktop. WSL2: wsl --status y wsl --update. Descargas: revise --progress=plain. TLS/PKIX: revise certificados y proxy. Permisos: compruebe UID 10001. La advertencia CDP es TECH-001 OPEN; el selector Cucumber es TECH-002 CLOSED. Chrome requiere --no-sandbox en este contenedor; el proceso usa UID 10001. CI conserva sólo reportes, XML, logs y capturas disponibles, más container.log; excluye clases y cachés.

La regla LF forma parte de main. v1.3.0 es la última release oficial y está publicada. La validación final confirmó BUILD SUCCESSFUL en local con headless=true y headless=false usando --offline, escenario Cucumber PASSED y Docker 6/6. Se observaron CDP local 153 frente a 152 y Docker 155 frente a 152; no reapareció el aviso del selector Cucumber/JUnit. quality-gate y docker-tests pasaron en el PR y main. La imagen base está fijada por digest, pero los paquetes apt pueden variar en builds futuros sin caché.

Docker en GitHub Actions

El workflow CI se activa con Pull Requests hacia main y pushes a main. quality-gate mantiene Java 21 y Gradle; docker-tests construye el Dockerfile y ejecuta Selenium/Cucumber con Chrome headless. No publica la imagen. Un fallo de pruebas deja el job Docker en Failure.

En GitHub abra Pull requests -> su PR -> Checks -> docker-tests -> Details para leer Build test image y Run headless tests and collect evidence. También puede abrir Actions -> CI -> ejecución y seleccionar ambos jobs. En Artifacts descargue docker-test-evidence cuando exista; incluye container.log, reportes, XML, logs y capturas disponibles. Se excluyen clases, cachés, temporales y binarios internos de Gradle. La carga se intenta aun ante fallos y conserva los archivos 14 días.

La validación registrada del PR y main confirmó quality-gate, docker-tests y ambos artifacts. v1.3.0 está publicada con estado Stable / Validated / Published.

5. Crear la primera automatización

1. Cree src/test/resources/features/login.feature:

```
# language: en
Feature: Login
  Scenario: Successful login
    Given the user opens the login page
    When the user signs in with "username" and "password"
    Then the dashboard is displayed
```

2. Cree LoginPage.java en pages. Mantenga locators privados y métodos de intención, por ejemplo open(), login() e isDashboardVisible().

3. Cree LoginSteps.java en steps. Obtenga el driver mediante DriverManager, delegue a LoginPage y use aserciones JUnit.

4. Configure la URL con -DbaseUrl=... y ejecute .\gradlew.bat test -Dheadless=true.

5. Revise el reporte HTML. Use esperas explícitas WebDriverWait; nunca Thread.sleep.

6. Reutilización paso a paso

1. Copie o clone el template; conserve la base original sin cambios.

2. Actualice rootProject.name en settings.gradle y group en build.gradle.

3. Configure baseUrl con -DbaseUrl=https://su-aplicacion, sin secretos en Git.

4. Abra la URL manualmente y ejecute una prueba smoke en headless.

5. Cuando el smoke funcione, sustituya el ejemplo por sus Features, Pages y Steps.

6. Añada tags como @smoke y @regression para seleccionar subconjuntos.

7. Inicialice Git, revise .gitignore, cree una rama y abra Pull Request.

8. Revise el workflow incluido en .github/workflows/ci.yml: ya ejecuta pruebas headless y publica las evidencias disponibles.

Criterio de salida: el equipo puede configurar URL, ejecutar una prueba y consultar el reporte sin editar componentes compartidos.

7. Contrato CI/CD, buenas prácticas y soporte

Workflow de GitHub Actions

El workflow .github/workflows/ci.yml ejecuta ./gradlew clean test -Dheadless=true con Java 21 de Temurin y el Gradle Wrapper en pull requests y pushes hacia main. En Windows, use .\gradlew.bat clean test -Dheadless=true para la validación local. Exit code 0 es éxito; cualquier otro código debe fallar el job. El contrato permite BROWSER, BASE_URL, HEADLESS y filtros cucumber.filter.tags; recolecte build/reports/cucumber/ (HTML y JSON), build/reports/tests/test, build/test-results/test, build/logs/automation.log y, sólo ante fallo, build/evidence/screenshots.

Flujo: checkout, Java 21 Temurin, Gradle Setup/Cache, Gradle Wrapper, pruebas headless, reportes y test-evidence. gradle/actions/setup-gradle@v6 usa caché básica para reutilizar dependencias e información de Gradle; puede reducir trabajo repetitivo en ejecuciones posteriores. Si no hay caché (cache miss), Gradle descarga lo necesario y las pruebas continúan. El Wrapper sigue siendo el mecanismo oficial. La caché no cambia Selenium, Cucumber, Page Object Model, features, steps ni el smoke test local.

El workflow usa ubuntu-latest y permisos mínimos de lectura. Intenta publicar las rutas anteriores como test-evidence durante 14 días incluso si Gradle falla. Una ruta vacía, como screenshots en una ejecución sin fallos, se ignora. Descargue el artifact en Artifacts dentro del resumen de GitHub Actions. La evidencia no cambia el estado PASS/FAIL del job. El job docker-tests ejecuta la imagen en el mismo workflow; no publica imágenes ni usa secretos nuevos.

Troubleshooting de Gradle

.\gradlew.bat --stop detiene Gradle Daemons ante problemas transitorios. Para diagnóstico de dependencias use .\gradlew.bat clean test --offline o .\gradlew.bat clean test -PjavaVersion=21 --offline; sólo usa caché y puede fallar si faltan dependencias. Si aparece PKIX path building failed o unable to find valid certification path, revise certificado Java, proxy, inspección SSL, red y daemon; ejecute java -version, .\gradlew.bat --version, .\gradlew.bat --stop y .\gradlew.bat clean test. No deshabilite SSL ni ignore certificados.

Regeneración del manual

scripts/create_manual.py requiere Python 3 y reportlab. Ejecute python scripts/create_manual.py desde la raíz. Genera manuales PDF en docs/ (español) y docs_en/ (inglés). work/ es temporal, está ignorada por Git, no es parte del framework y puede eliminarse.

Buenas prácticas

Un Page Object por pantalla/componente; nombres descriptivos; configuración externa; locators estables; datos aislados; screenshots y logs como evidencia; commits pequeños; ramas, Pull Requests y revisión de código; actualización deliberada de dependencias.

Para problemas de Java, navegador, features o CI, consulte docs/TROUBLESHOOTING.md y conserve el mensaje completo antes de modificar código.

8. CI/CD con GitHub Actions

CI (integración continua) valida automáticamente cambios antes de integrarlos. Detecta temprano errores de compilación y pruebas y ofrece al equipo un resultado compartido. CD significa entrega continua (preparar una versión para publicar) o despliegue continuo (publicarla automáticamente). Este proyecto utiliza principalmente CI; no despliega aplicaciones.

Qué ejecuta GitHub Actions

GitHub Actions lee .github/workflows/ci.yml cuando se abre o actualiza un Pull Request hacia main y cuando hay un push a main. Un Pull Request o PR es una propuesta de unir una rama de trabajo con otra. El workflow obtiene el código (checkout), prepara Java 21 Temurin y caché Gradle en un runner Linux ubuntu-latest, y ejecuta el Gradle Wrapper con Selenium y Cucumber en modo headless, sin ventana visible.

Flujo para principiantes

Developer -> feature branch -> commit -> push -> Pull Request hacia main -> GitHub Actions -> quality-gate -> Gradle -> Selenium + Cucumber -> PASS/FAIL -> evidencias -> revisión -> merge a main.

El workflow se ejecuta automáticamente. No hay botón de ejecución manual configurado. El merge debe esperar la revisión y un check exitoso; el ruleset activo de main exige un Pull Request, la rama actualizada y quality-gate aprobado; un fallo bloquea el merge.

9. Cómo ejecutar y revisar el CI/CD en GitHub Actions

Necesita Git, acceso al repositorio en GitHub y permiso para enviar ramas. Ejecute desde la raíz del proyecto.

1. Cree una rama separada de main: git switch -c feature/mi-cambio. Si ya existe: git switch feature/mi-cambio. Main es la línea compartida; una feature branch aísla su trabajo.

2. Modifique el proyecto, por ejemplo un Page Object y sus pruebas.

3. Revise git status y git diff. Compruebe que no haya credenciales ni archivos generados.

4. Ejecute las pruebas locales:

```
Windows PowerShell: .\gradlew.bat clean test "-Dheadless=true"
Linux/macOS: ./gradlew clean test -Dheadless=true
```

Espere BUILD SUCCESSFUL. Si ve BUILD FAILED, corrija el error.

Continuación: Pull Request y revisión de resultados

5. Guarde el cambio en Git: git add ruta/del/archivo y git commit -m "Describe mi cambio". Un commit es una instantánea identificable.

6. Envíe la rama: git push -u origin feature/mi-cambio la primera vez; luego git push. Push copia los commits a GitHub.

7. En el repositorio de GitHub abra Pull requests -> New pull request. Seleccione main como base (destino) y feature/mi-cambio como compare (origen). Revise el contenido, agregue título y descripción y pulse Create pull request. No haga merge todavía.

8. GitHub detecta el PR y ejecuta .github/workflows/ci.yml automáticamente. Cada nuevo commit seguido de push a la misma rama actualiza el PR y inicia otra ejecución.

Desde el Pull Request

Abra Repositorio -> Pull requests -> su PR -> Checks; según la interfaz, los checks también aparecen en Conversation. Busque quality-gate y docker-tests; pulse Details en cada uno. Allí verá checkout, Java 21, caché Gradle, pruebas, carga de evidencias y resultado final. Expanda un step para ver sus logs.

Desde Actions

Abra Repositorio -> Actions -> workflow CI -> ejecución del PR. Identifique rama, PR, commit, fecha, estado y duración. Abra la ejecución y ambos jobs, quality-gate y docker-tests, para inspeccionar cada step.

Estado visible	Qué significa
Success (verde)	Las verificaciones terminaron correctamente.
Failure (rojo)	Una verificación falló.
In progress (amarillo/progreso)	La ejecución todavía no termina.
Cancelled	La ejecución se canceló.
Skipped	Un step no se ejecutó por una condición.

Lea el texto del estado además del color: así puede interpretar el resultado aun sin distinguir colores.

10. ¿Qué es un Quality Gate?

Es una puerta de control: las verificaciones automáticas deben terminar bien antes de considerar listo un cambio. El job/check estable se llama quality-gate. Gradle devuelve exit code 0 cuando compilación y pruebas pasan; un código distinto de cero indica fallo. GitHub muestra PASS/Success para continuar a revisión o FAIL/Failure para corregir. La carga posterior de artifacts no cambia FAIL a PASS.

¿Qué hacer si el Quality Gate falla?

1. Abra el PR -> Checks -> quality-gate -> Details.

2. Localice el step Failure, expándalo y lea el error y el resumen final.

3. Descargue test-evidence si existe; revise reportes, XML, logs y screenshots. Si Gradle terminó antes de generar archivos, algunos faltarán.

4. Corrija el problema en su rama y repita la prueba local:

```
.\gradlew.bat clean test "-Dheadless=true"
```

En Linux/macOS use ./gradlew clean test -Dheadless=true.

5. Revise git diff, haga otro git add y git commit, y ejecute git push. El PR se actualiza y Actions ejecuta de nuevo quality-gate. Espere el nuevo resultado.

Cómo descargar las evidencias de GitHub Actions

Un artifact es un archivo descargable creado por una ejecución y separado del código. Abra Repository -> Actions -> CI -> ejecución del PR -> Artifacts -> test-evidence. Descargue y extraiga el ZIP. Se conserva 14 días. Se intenta publicar tanto en PASS como en FAIL, pero sólo aparece cuando existen archivos. Si falta, revise el step Upload test evidence y si Gradle creó resultados. Las capturas suelen faltar cuando no fallan escenarios.

Ruta local / artifact	Qué contiene
build/reports/cucumber/cucumber.html	Escenarios y pasos Cucumber.
build/reports/cucumber/cucumber.json	Resultado estructurado Cucumber y attachments.
build/reports/tests/test/	Reporte HTML Gradle/JUnit.
build/test-results/test/	Resultados XML estructurados.
build/logs/automation.log	Log cronológico.
build/evidence/screenshots/	Capturas de fallos cuando correspondan.

11. Protección de la rama main

Las reglas de GitHub controlan cómo se integra código en main. El flujo recomendado es rama de trabajo, Pull Request, quality-gate, revisión y merge. Un status check fallido o pendiente impide el merge cuando está configurado como obligatorio.

El ruleset de main documentado para este repositorio requiere Pull Request, quality-gate aprobado y rama actualizada; también bloquea force push y eliminación de main. Consulte el estado actual en Repository > Settings > Rules > Rulesets antes de integrar cambios.

Si un PR falla, abra Checks > quality-gate > Details, revise el paso fallido y descargue test-evidence cuando exista. Corrija el cambio en la rama, repita las pruebas locales y envíe otro commit. GitHub vuelve a ejecutar el workflow.

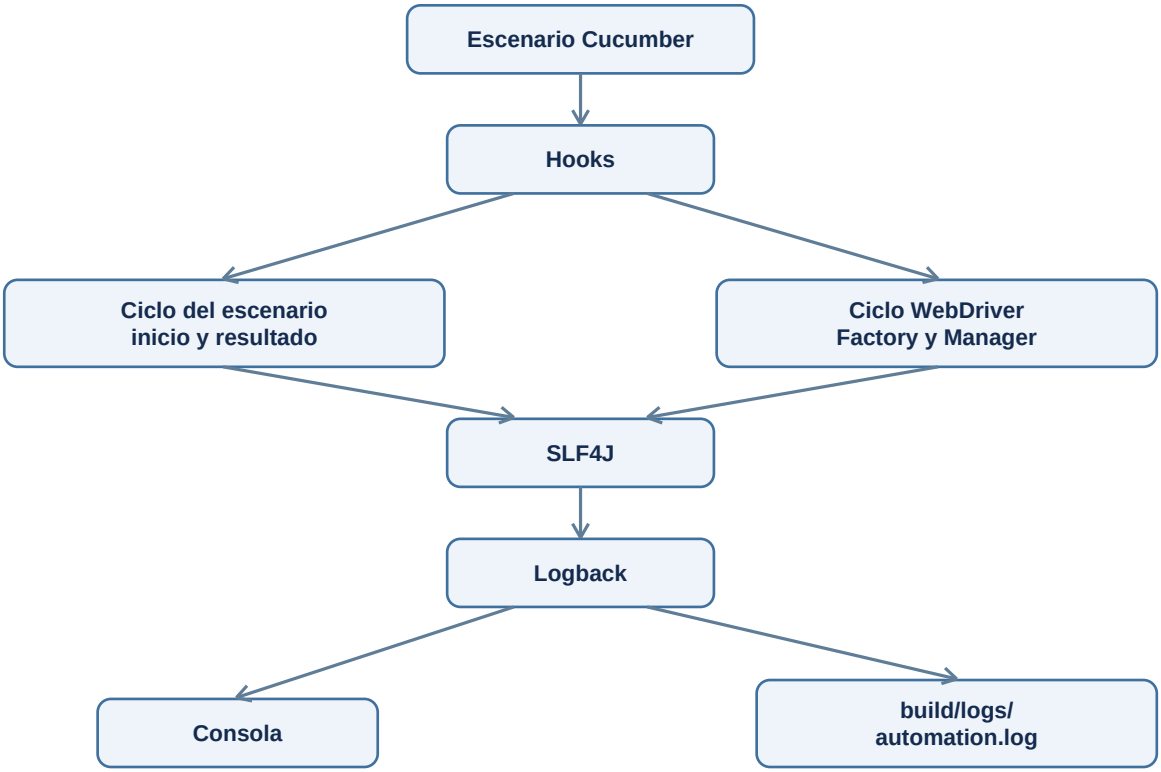
La versión v1.1.0 se publicó el 1 de octubre de 2026. El historial de la página inicial contiene las fechas de v1.1.0 y v1.2.0. Consulte los README para el estado vigente.

12. Logging y observabilidad básica

Los eventos técnicos permiten reconstruir la configuración efectiva, el resultado del escenario y el cierre de WebDriver. Consulte el archivo de log junto con los reportes para diagnosticar un fallo.

El logging ayuda a reconstruir el inicio, la configuración efectiva, el resultado y el cierre de un escenario cuando una prueba falla o se ejecuta en CI/CD. El código usa SLF4J 2.0.20 como API común para Hooks y driver, evitando configurar salidas en cada clase. Logback 1.6.5 es el proveedor de pruebas que centraliza niveles, formato y destinos en src/test/resources/logback-test.xml.

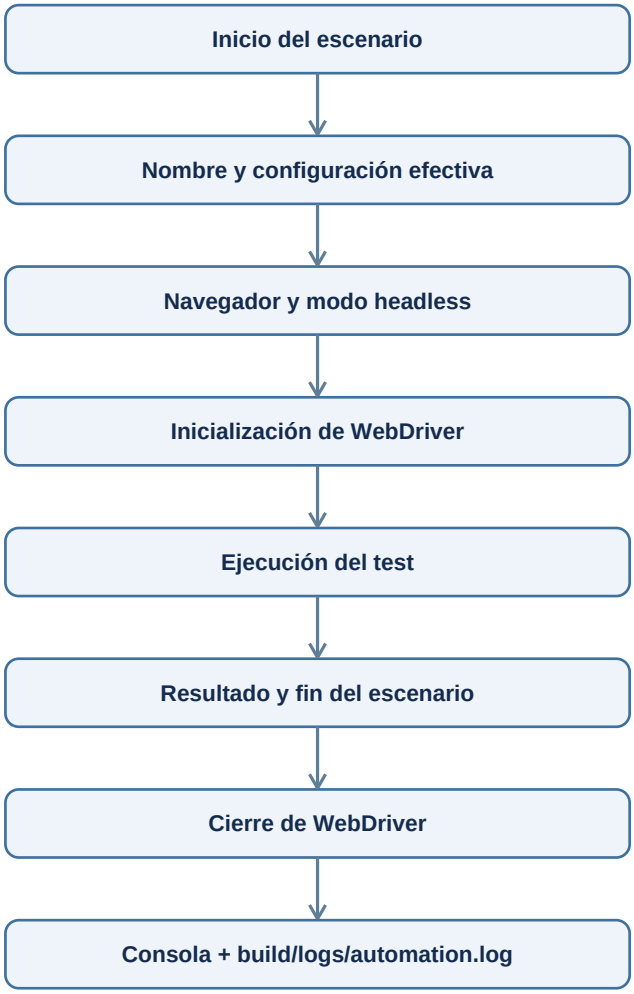
Diagrama A - Arquitectura de logging



Los eventos del framework pasan por SLF4J y Logback. Cada evento se emite a consola y a build/logs/automation.log; no se duplican llamadas de logging en Steps o Page Objects.

12.1 Flujo de observabilidad

Diagrama B - Eventos en orden de ejecución



Los Hooks registran inicio, nombre, navegador y headless efectivos antes de crear el driver. DriverFactory registra la inicialización; al finalizar, el Hook registra el estado Cucumber y DriverManager cierra el WebDriver. La consola y el archivo reciben los eventos a medida que ocurren.

12.2 Niveles de logging

Nivel	Uso
DEBUG	Detalle técnico de creación del driver; oculto por defecto.
INFO	Eventos normales: escenario, configuración segura y WebDriver.
WARN	Condición recuperable, como ausencia de soporte para capturas.
ERROR	Fallo al cerrar WebDriver o manejar evidencia existente.

12.3 Ejecución y seguridad

Ejecute .\gradlew.bat clean test -Dheadless=true en Windows, o ./gradlew clean test -Dheadless=true en GitHub Actions. Gradle muestra los logs de test en consola. Logback guarda el archivo bajo build/, ignorado por Git. gradlew clean elimina el log anterior y la siguiente ejecución crea uno nuevo. El workflow existente ya incluye build/logs/ en el artifact test-evidence; no se modificó el workflow.

Extracto abreviado de una ejecución headless real

```
2026-10-01 17:36:35.726 INFO [Test worker] Hooks - Starting scenario: Validar el encabezado de la
página de ejemplo
2026-10-01 17:36:35.734 INFO [Test worker] Hooks - Scenario configuration: browser=CHROME,
headless=true
2026-10-01 17:36:37.110 INFO [Test worker] Hooks - Finished scenario: Validar el encabezado de la
página de ejemplo, status=PASSED
2026-10-01 17:36:37.208 INFO [Test worker] DriverManager - WebDriver closed.
```

No registre secretos, tokens, contraseñas, cookies, headers confidenciales ni propiedades indiscriminadamente. baseUrl no se registra porque una URL arbitraria puede incluir datos sensibles. Las rutas de error existentes registran excepciones completas; las de herramientas externas podrían contener datos de la aplicación. Revise las trazas antes de compartir logs o artifacts.

Si falta automation.log, confirme que test inició el proceso de pruebas y consulte build/logs/. Si falta DEBUG, revise el nivel INFO predeterminado en logback-test.xml. Si hay avisos de proveedores SLF4J, inspeccione el classpath con .\gradlew.bat dependencies --configuration testRuntimeClasspath. Si falla la descarga por PKIX, revise certificados y proxy del JDK sin desactivar TLS.

12.4 Diagnóstico del logging

Si el archivo no aparece, confirme que la tarea test alcanzó la ejecución de escenarios y que src/test/resources/logback-test.xml está en el classpath. clean elimina logs anteriores. El nivel raíz INFO oculta DEBUG; un fallo de compilación previo a test puede impedir la creación del archivo.

13. Captura automática de evidencias

Una evidencia visual conserva el estado de la página cuando falla un escenario Cucumber. Ayuda a investigar un error que no puede reconstruirse sólo con el mensaje de la aserción. La captura se intenta únicamente si el escenario falló y screenshotOnFailure=true; un escenario exitoso no crea un PNG de fallo.

Cómo se genera

El Hook @After consulta el estado del escenario y entrega el WebDriver activo a EvidenceManager antes de cerrarlo. El gestor verifica que el driver exista, que una sesión RemoteWebDriver siga activa y que soporte TakesScreenshot. Captura bytes PNG, crea build/evidence/screenshots/, guarda el archivo y adjunta los mismos bytes mediante Scenario.attach(..., "image/png", "failure-screenshot").

Paso	Resultado
Escenario fallido	Activa la captura cuando screenshotOnFailure=true.
WebDriver disponible	Comprueba sesión activa y soporte de screenshots.
PNG local	Guarda en build/evidence/screenshots/.
Attachment	Incluye image/png en el resultado Cucumber.
Cierre	El Hook libera WebDriver incluso si la captura falla.

Consulta y correlación

Abra build/reports/cucumber/cucumber.html y busque el escenario fallido y su attachment. El JSON en build/reports/cucumber/cucumber.json conserva la misma evidencia para procesamiento automatizado. Compare el nombre y estado del escenario con build/logs/automation.log y el PNG en build/evidence/screenshots/. El nombre del archivo usa un nombre seguro de hasta 80 caracteres, fecha/hora y UUID.

Si la captura falla

Si falta el driver, la sesión está cerrada o la captura devuelve bytes vacíos, se registra un warning y se conserva el fallo original. Si no puede guardarse el archivo, todavía se intenta adjuntar la imagen al escenario; si falla el attachment, el escenario sigue fallido. La ausencia de PNG en un PASS es normal.

Ejemplo y privacidad

Ejecute .\gradlew.bat clean test -Dheadless=true y consulte el HTML. Para comprobar una captura, use una prueba controlada que falle y restáurela antes de volver a ejecutar la suite. clean elimina la evidencia anterior: inspecciónela antes de repetir. Las capturas y excepciones pueden incluir datos visibles de la aplicación; revíselas antes de compartir artifacts. build/ está ignorado por Git.

14. Generación y consulta de reportes

Los reportes muestran qué escenarios y pasos se ejecutaron, cuáles pasaron o fallaron y qué evidencia se adjuntó. El runner JUnit Platform configura los plugins nativos de Cucumber pretty, HTML y JSON; Gradle produce HTML y JUnit XML. No hay dashboard histórico ni agregación entre ejecuciones.

Generar y abrir resultados

Desde la raíz, ejecute .\gradlew.bat clean test -Dheadless=true en PowerShell o ./gradlew clean test -Dheadless=true en Linux/macOS. Después abra build/reports/cucumber/cucumber.html y build/reports/tests/test/index.html en un navegador. Use cucumber.json y los XML de build/test-results/test/ para procesamiento o CI. pretty muestra los pasos en consola.

Salida	Ruta y uso
Cucumber HTML	build/reports/cucumber/cucumber.html - escenarios, pasos y attachments.
Cucumber JSON	build/reports/cucumber/cucumber.json - resultados estructurados y attachments.
Gradle HTML	build/reports/tests/test/ - resultado técnico de la tarea test.
JUnit XML	build/test-results/test/ - resultados estructurados para CI/CD.
Logback	build/logs/automation.log - eventos técnicos.
Screenshot PNG	build/evidence/screenshots/ - imagen de un fallo elegible.

Interpretar un fallo

Busque el escenario y el paso fallido en Cucumber HTML/JSON. Compare su nombre y estado con el log y el resultado JUnit XML. Si el navegador estaba disponible y la captura estaba activada, abra el attachment image/png o el PNG local. Un error previo a test puede dejar reportes ausentes; una captura fallida no reemplaza el resultado del escenario.

GitHub Actions y límites

El job quality-gate ejecuta pruebas headless. Upload test evidence usa if: always() y recoge los archivos disponibles bajo build/reports/, build/test-results/, build/logs/ y build/evidence/screenshots/ en el artifact test-evidence durante 14 días. Un artifact puede estar incompleto si el build falló temprano. clean reemplaza resultados locales; conserve lo necesario antes de repetir y revise datos sensibles antes de compartir.

15. Glosario

Términos clave del template y de la automatización Web UI.

Término	Definición
Automation Testing	Ejecución de pruebas mediante software para validar una aplicación.
Selenium	Biblioteca que automatiza navegadores web.
WebDriver	Interfaz que controla Chrome o Edge desde el código.
Cucumber	Herramienta BDD que ejecuta escenarios escritos en Gherkin.
BDD	Enfoque que describe el comportamiento esperado antes de implementarlo.
Gherkin	Lenguaje legible para escribir Features, Scenarios y Steps.
Feature	Archivo o capacidad de negocio descrita en Gherkin.
Scenario	Caso concreto que valida un comportamiento esperado.
Step Definition	Código Java que implementa una frase de Gherkin.
Page Object Model	Patrón que encapsula la UI y sus interacciones en clases Page.
Hooks	Código que prepara y limpia el estado antes o después de escenarios.
Runner de Cucumber	Clase que configura el descubrimiento y la ejecución de Cucumber.
Gradle	Herramienta de compilación, dependencias y ejecución de pruebas.
Gradle Wrapper	Scripts versionados que ejecutan la versión requerida de Gradle.
JUnit Platform	Plataforma que descubre y coordina la ejecución de pruebas.
Headless	Ejecución del navegador sin mostrar una ventana gráfica.
CI/CD	Integración y entrega continua mediante una pipeline automatizada.
Pipeline	Secuencia automatizada de compilación, pruebas y publicación.
Git	Sistema de control de versiones distribuido.
GitHub	Plataforma para repositorios, revisiones y workflows de CI.
Locator	Regla que permite identificar un elemento en la interfaz.
XPath	Sintaxis para ubicar elementos a partir de la estructura del DOM.
CSS Selector	Sintaxis para ubicar elementos con atributos o clases CSS.
Assertion	Verificación que confirma el resultado esperado de una prueba.
Test Data	Datos controlados utilizados durante la ejecución de pruebas.
Environment Variable	Valor externo al código que configura una ejecución.
Explicit Wait	Espera controlada hasta que una condición de WebDriver se cumple.

Glosario (continuación)

Término	Definición
Screenshot	Captura visual utilizada como evidencia, especialmente ante fallos.
CI	Integración continua: valida cambios automáticamente antes de integrarlos.
CD	Entrega o despliegue continuo; aquí no hay despliegue automático.
GitHub Actions	Servicio de GitHub que ejecuta workflows.
Workflow	Archivo de instrucciones automáticas, como ci.yml.
Job	Grupo de pasos ejecutado en un runner.
Step	Instrucción individual de un job.
Runner de GitHub Actions	Equipo temporal que ejecuta un job.
ubuntu-latest	Etiqueta de un runner Linux Ubuntu en GitHub Actions.
Pull Request	Propuesta de integrar una rama después de revisión.
Branch	Línea de trabajo separada dentro de Git.
main	Rama principal compartida del repositorio.
Feature Branch	Rama dedicada a un cambio concreto.
Quality Gate	Verificación que debe pasar antes de considerar listo un cambio.
Status Check	Resultado visible de una verificación de GitHub.
Required Status Check	Check que debe pasar para permitir el merge.
Branch Protection	Reglas que protegen una rama ante integraciones indebidas.
Artifact	Archivo descargable producido por una ejecución.
Log	Registro cronológico de mensajes de ejecución.
Report	Resumen legible de resultados de pruebas.
Exit Code	Número de salida: 0 éxito; otro valor, fallo.
PASS	La verificación terminó correctamente.
FAIL	La verificación terminó con un error.
Commit	Instantánea identificable de cambios en Git.
Push	Envío de commits locales al repositorio remoto.
Merge	Unión de cambios de una rama a otra.
Checkout	Obtención del código de una revisión para ejecutarlo.